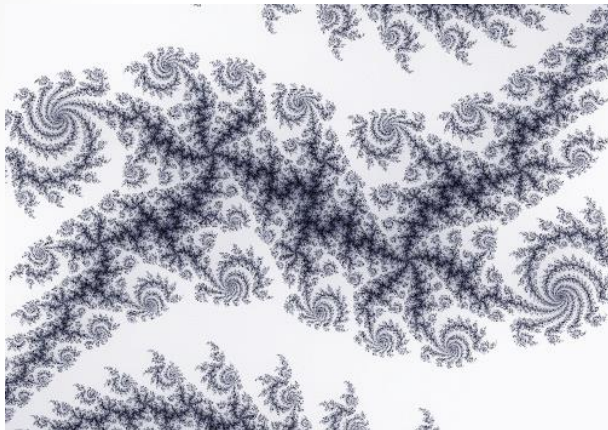
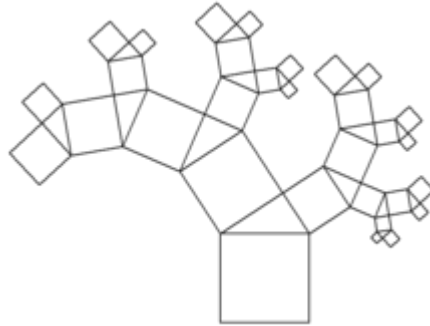


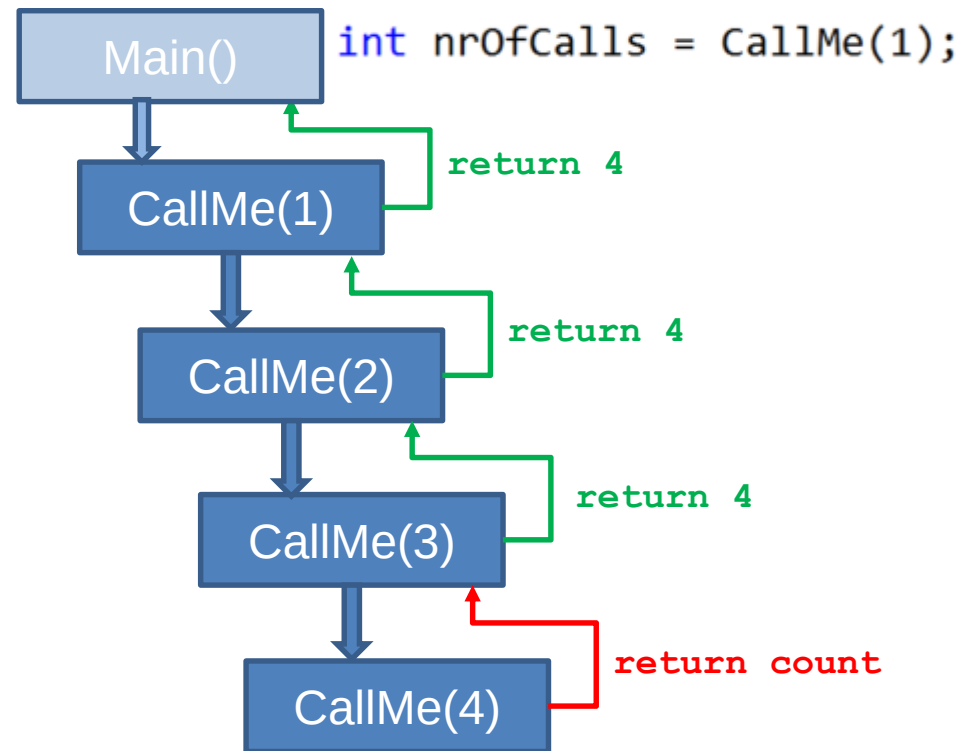
Rekursion



Rekursion

Bei der **rekursiven Programmierung** ruft sich eine **Methode** (Prozedur oder Funktion) selbst wieder auf.

```
static int CallMe(int count)
{
    Console.WriteLine(count + " ");
    if (count == 4)
    {
        return count;
    }
    else
    {
        return CallMe(++count);
    }
}
```



Wesentliche Merkmale rekursiver Programmierung:

- **Abbruchbedingung!**
- Man soll sich in jeder Rekursionsstufe mehr der Abbruchbedingung nähern - das **zu lösende Problem** soll **kleiner** werden.

Beispiel 1: Fakultätsberechnung

Die **Fakultät** (auch **Faktorielle** genannt) ist eine mathematische Funktion:

$$\text{Fakultät}(n) = 1 * 2 * \dots * n$$

$$\text{Fakultät}(0) = 1$$

Beispiel:

$$5! = 1 * 2 * 3 * 4 * 5 = 120$$

n	$n!$
0	1
1	1
2	2
3	6
4	24
5	120
10	3.628.800
20	$2,432... \cdot 10^{18}$
50	$3,041... \cdot 10^{64}$
100	$9,332... \cdot 10^{157}$

Faktorielle: Iterative Lösung

Die Methode „**Factorial**“ ermittelt zum Parameter „***n***“ das Produkt aller Zahlen zwischen 1 und ***n***.

```
static long Factorial(int n)
{
    long factorial = 1;
    for (int i = 2; i <= n; i++)
    {
        factorial = factorial * i;
    }
    return factorial;
}
```

Faktorielle: Rekursiver Ansatz

Beispiel: $75! = 75 * 74 * 73 * \dots * 2 * 1$

$$75! = 75 * 74!$$

$74! = 74 * 73 * 72 * \dots * 2 * 1$

$$74! = 74 * 73!$$

Rekursive Definition:

Factorial(n) := n * ***Factorial***(n-1) für n > 0 → Abstieg

Factorial(0) := 1 → Ausstieg

Faktorielle: Rekursive Lösung

Aufgabe:

Entwirf den rekursiven Algorithmus für die Methode

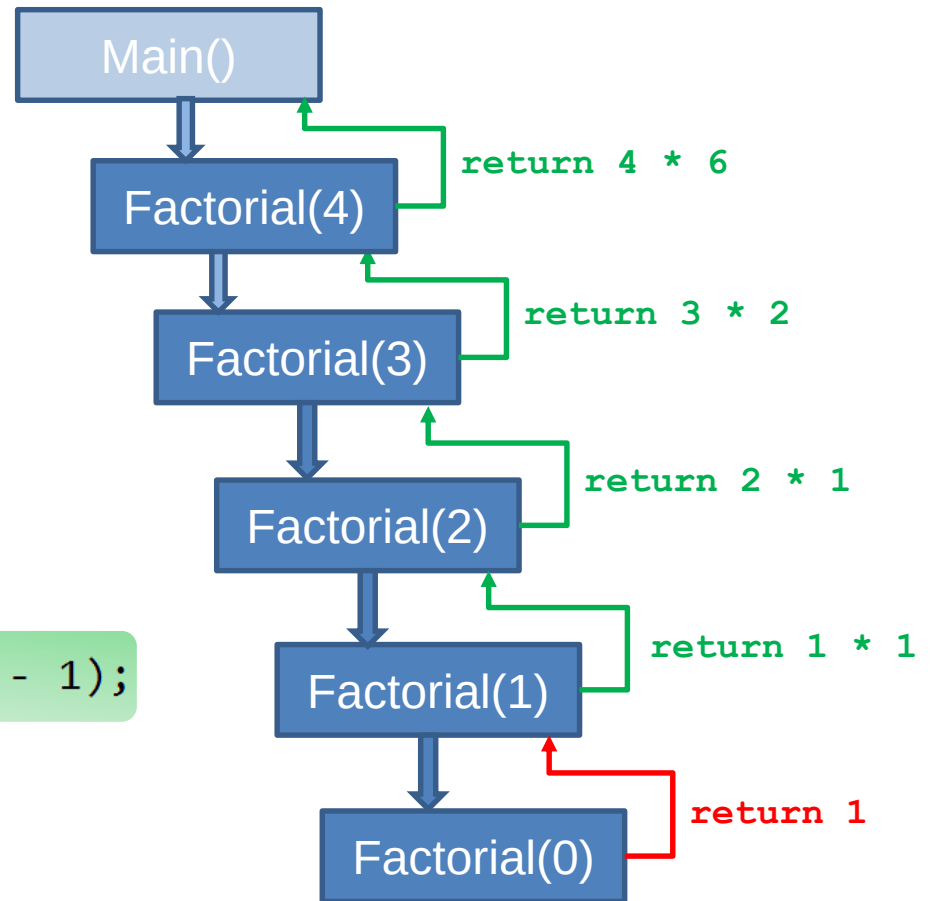
long Factorial(int n)

```
static long Factorial(int n)
{
    if (n == 0)
    {
        return 1;
    }
    else
    {
        return n * Factorial(n - 1);
    }
}
```

Faktorielle Rekursiv: Ablauf

Beispiel: $4! = 4 * 3 * 2 * 1 = 24$

```
static long Factorial(int n)
{
    if (n == 0)
    {
        return 1;
    }
    else
    {
        return n * Factorial(n - 1);
    }
}
```



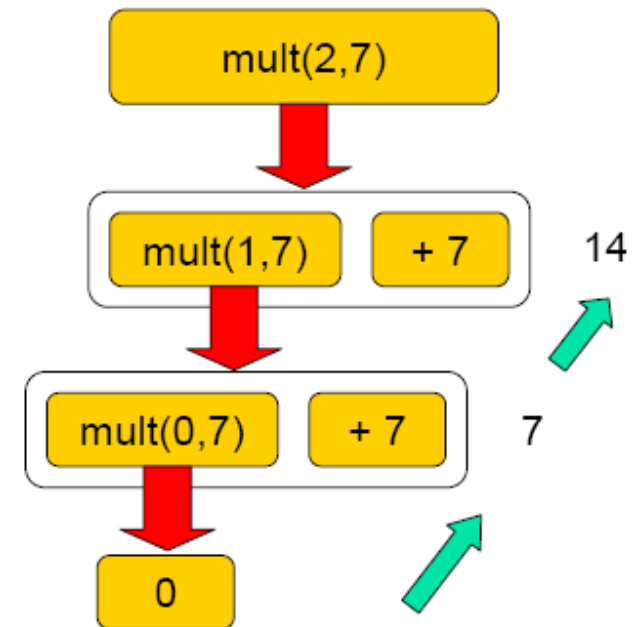
Beispiel 2: Multiplikation

Aufgabe:

- Entwirf einen rekursiven Algorithmus für die Multiplikation zweier Operanden a und b.
- Dabei soll der Operand b a-Mal mit sich selbst **addiert** werden.

```
static int Multiply(int a, int b)
{
    if (a == 0)
    {
        return 0;
    }
    else
    {
        return b + Multiply(a - 1, b);
    }
}
```

```
int product = Multiply(a, b);
```



Beispiel 3: Fibonacci-Folge

Die Fibonacci-Folge $f_1, f_2, f_3, \dots$ ist durch das **rekursive** Bildungsgesetz

$$f_n = f_{n-1} + f_{n-2} \quad \text{für } n > 2$$

mit den Anfangswerten

$$f_1 = f_2 = 1$$

definiert. Das bedeutet in Worten:

- Für die beiden ersten Zahlen wird der Wert *eins* vorgegeben.
- Jede weitere Zahl ist die Summe ihrer beiden Vorgänger in der Folge.

0, 1, 1, 2, 3, 5, 8, 13, ...
(optional) 0+1, 1+1, 1+2, 2+3, 3+5, 5+8, ...

Aufgabe:

Entwirf den rekursiven Algorithmus für die Methode

```
int Fibonacci(int n)
```

